Heart Disease Prediction
using Supervised Machine Learning

CAPSTONE #3

M2M TECH CONNECT – DATA TALENT PROGRAM (COHORT 18)

BY: ANGELO RAMIREZ

NOVEMBER 2025

Objectives

- Use a dataset from Kaggle named “Heart Failure Dataset” to predict if one has “Heart Disease” based on the columns/features
- Analyze and clean the dataset to get it ready for model training
- Make a machine learning pipeline using Scikit Learn to:
  - A. Scale numerical columns
  - B. Convert categorical columns using “One Hot Encoder”
  - C. Use “**Random Forest Classifier**” as our algorithm for Supervised Machine Learning
- Select the best Hyper-Parameters using “Grid Search CV” and train the model
- Test and evaluate the model

```
1 df = pd.read_csv('heart.csv')
2 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype  
---  ------          --------------  -----  
 0   Age             918 non-null    int64  
 1   Sex             918 non-null    object 
 2   ChestPainType   918 non-null    object 
 3   RestingBP       918 non-null    int64  
 4   Cholesterol     918 non-null    int64  
 5   FastingBS       918 non-null    int64  
 6   RestingECG      918 non-null    object 
 7   MaxHR           918 non-null    int64  
 8   ExerciseAngina  918 non-null    object 
 9   Oldpeak         918 non-null    float64
 10  ST_Slope        918 non-null    object 
 11  HeartDisease    918 non-null    int64  
dtypes: float64(1), int64(6), object(5)
memory usage: 86.2+ KB
```

Dataset Overview

- Heart Failure Dataset
- Source: https://www.kaggle.com/datasets/tan5577/heart-failure-dataset/data
- Num. of rows: 918
- Num. of columns: 12
- The dataset does not have empty values

Column Name	Type	Description
Age	Integer	age of the patient [years]
Sex	Categorical	sex of the patient [**M**: Male, **F**: Female]
Chest Pain Type	Categorical	chest pain type [**TA**: Typical Angina, **ATA**: Atypical Angina, **NAP**: Non-Anginal Pain, **ASY**: Asymptomatic]
Resting BP	Integer	resting blood pressure [mm Hg]
Cholesterol	Integer	serum cholesterol [mm/dl]
Fasting BS	Integer	fasting blood sugar [1: if Fasting BS > 120 mg/dl, 0: otherwise]
Resting ECG	Categorical	resting electrocardiogram results [**Normal**: Normal, **ST**: having ST-T wave abnormality (T wave inversions and/or ST elevation or depression of > 0.05 mV), **LVH**: showing probable or definite left ventricular hypertrophy by Estes' criteria].
Max HR	Integer	maximum heart rate achieved [Numeric value between 60 and 202]
Exercise Angina	Categorical	exercise-induced angina [**Y**: Yes, **N**: No]
Old Peak	Float	ST [Numeric value measured in depression]
ST Slope	Categorical	the slope of the peak exercise ST segment [**Up**: upsloping, **Flat**: flat, **Down**: down sloping]
Heart Disease	Integer	output class [**1**: heart disease, **0**: Normal]

Dataset Overview

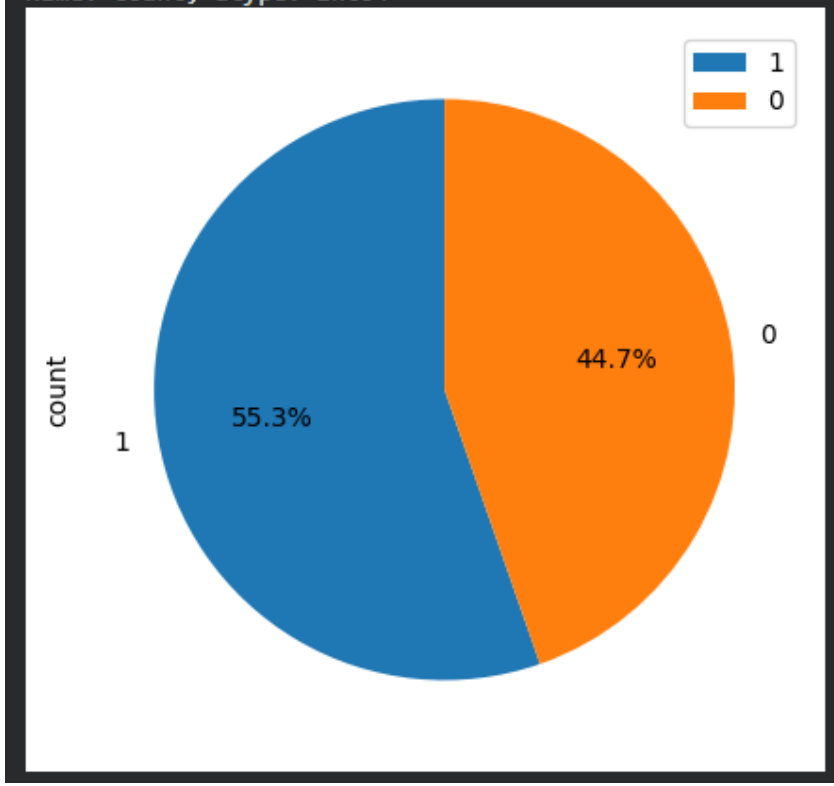
Numerical: 'Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak' (6 columns)

Categorical: 'Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope' (5 columns)

Output: 'Heart Disease'

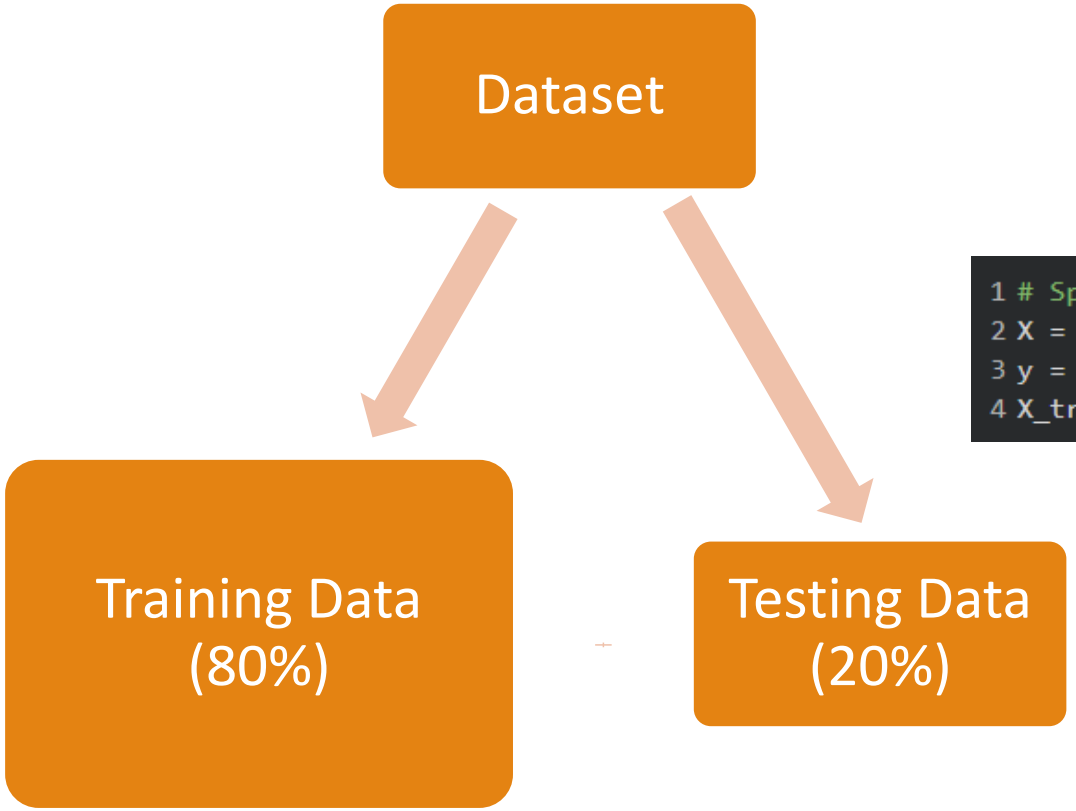
```
1 hd_count = df['HeartDisease'].value_counts()
2 hd_count.plot.pie(legend=True, figsize=(5,5), autopct='%1.1f%%', startangle=90)
3 print(hd_count)
```

```
HeartDisease
1    508
0    410
Name: count, dtype: int64
```



Heart Disease Data Distribution

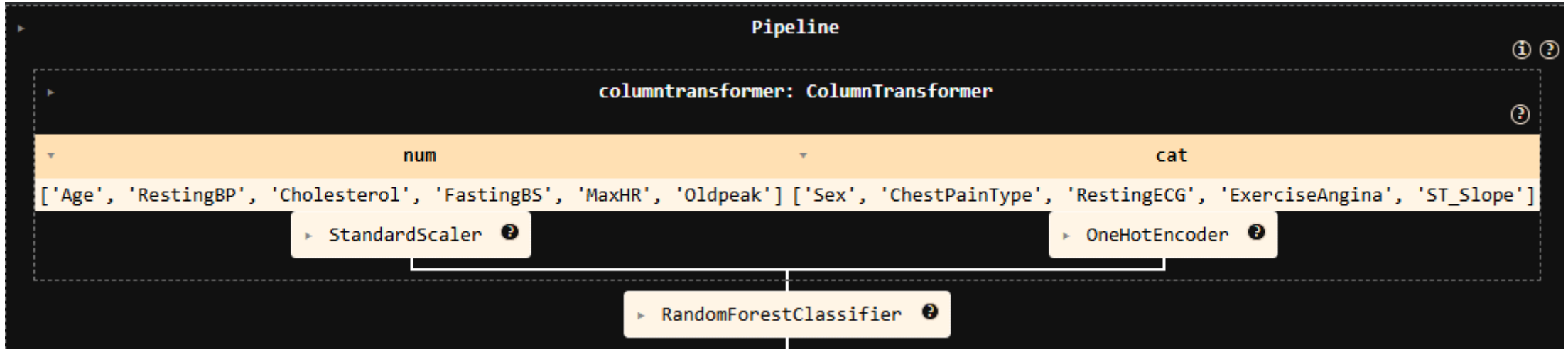
Splitting the Dataset




```
# Split data into training and testing sets
X = df.drop('HeartDisease', axis=1)
y = df['HeartDisease']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Machine Learning Pipeline

- Pipeline allows you to sequentially apply a list of transformers to preprocess the data and, if desired, conclude the sequence with a final predictor for predictive modeling.
- Source: https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html



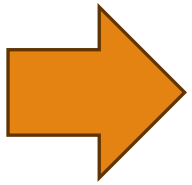
Standard Scaler

- Used for numerical features ('Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak')
- Standard Scaler is a feature scaling technique which follows Standard Normal Distribution (SND) and is used to standardize the values of numeric features. It transforms data so that the mean becomes 0 and the standard deviation becomes 1.
- Standardization of datasets is a common requirement for many machine learning estimators implemented in scikit-learn; they might behave badly if the individual features do not more or less look like standard normally distributed data

One Hot Encoder

- Used for categorical features ('Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope')
- One Hot Encoding is a method for converting categorical variables into a binary format. It creates new columns for each category where 1 means the category is present and 0 means it is not. The primary purpose of One Hot Encoding is to ensure that categorical data can be effectively used in machine learning models.

Chest Pain Type
TA
ATA
NAP
ASY

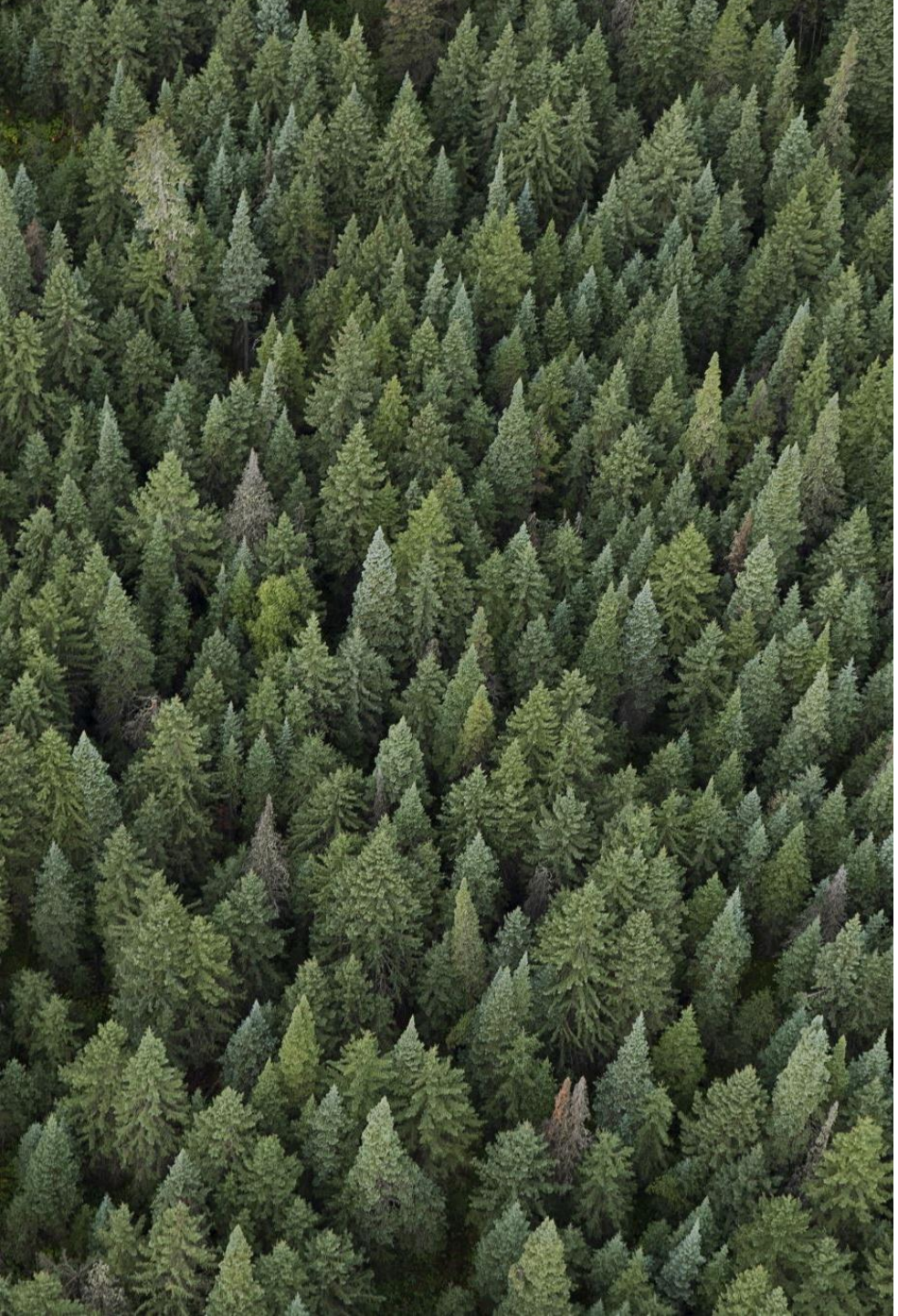


ChestPainType_TA	ChestPainType_ATA	ChestPainType_NAP	ChestPainType_ASY
0	0	0	0
1	1	1	1

Feature Names after Preprocessing

```
# Get the feature names after transformation
feature_names = best_model_pipeline[0].get_feature_names_out()
feature_names
```

```
array(['num__Age', 'num__RestingBP', 'num__Cholesterol', 'num__FastingBS',
       'num__MaxHR', 'num__Oldpeak', 'cat__Sex_F', 'cat__Sex_M',
       'cat__ChestPainType_ASY', 'cat__ChestPainType_ATA',
       'cat__ChestPainType_NAP', 'cat__ChestPainType_TA',
       'cat__RestingECG_LVH', 'cat__RestingECG_Normal',
       'cat__RestingECG_ST', 'cat__ExerciseAngina_N',
       'cat__ExerciseAngina_Y', 'cat__ST_Slope_Down',
       'cat__ST_Slope_Flat', 'cat__ST_Slope_Up'], dtype=object)
```



Random Forest Classifier

- A Random Forest is an ensemble machine learning model that combines multiple decision trees. Each tree in the forest is trained on a random sample of the data and considers only a random subset of features when making splits.
- Random Forests give accurate results and are less likely to overfit than single decision trees.

```
1 list(model_pipeline.get_params().keys())
```

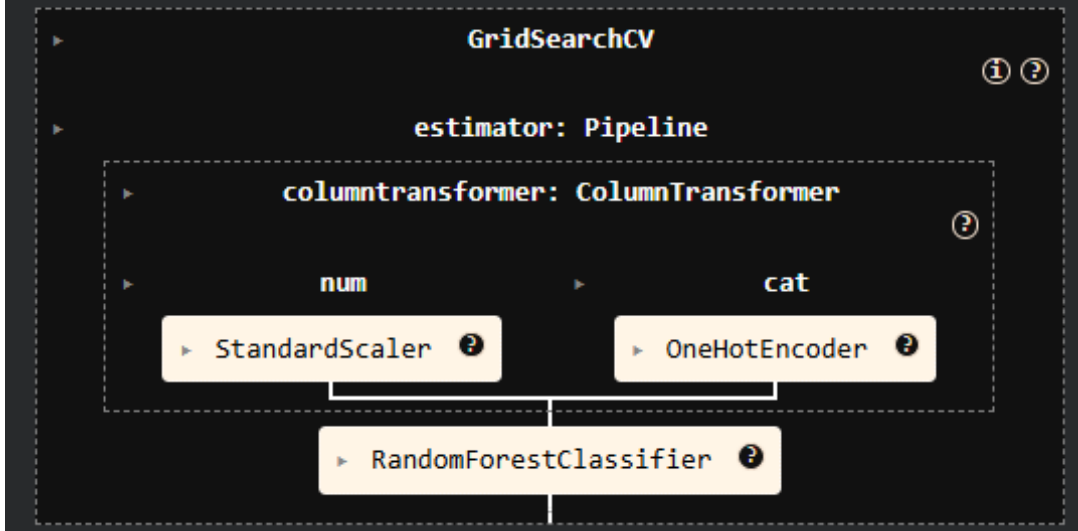
```
['memory',
 'steps',
 'transform_input',
 'verbose',
 'columntransformer',
 'randomforestclassifier',
 'columntransformer__force_int_remainder_cols',
 'columntransformer__n_jobs',
 'columntransformer__remainder',
 'columntransformer__sparse_threshold',
 'columntransformer__transformer_weights',
 'columntransformer__transformers',
 'columntransformer__verbose',
 'columntransformer__verbose_feature_names_out',
 'columntransformer__num',
 'columntransformer__cat',
 'columntransformer__num__memory',
 'columntransformer__num__steps',
 'columntransformer__num__transform_input',
 'columntransformer__num__verbose',
 'columntransformer__num__standardscaler',
 'columntransformer__num__standardscaler__copy',
 'columntransformer__num__standardscaler__with_mean',
 'columntransformer__num__standardscaler__with_std',
 'columntransformer__cat__memory',
 'columntransformer__cat__steps',
 'columntransformer__cat__transform_input',
 'columntransformer__cat__verbose',
 'columntransformer__cat__onehotencoder',
 'columntransformer__cat__onehotencoder__categories',
 'columntransformer__cat__onehotencoder__drop',
 'columntransformer__cat__onehotencoder__dtype',
 'columntransformer__cat__onehotencoder__feature_name_combiner',
 'columntransformer__cat__onehotencoder__handle_unknown',
 'columntransformer__cat__onehotencoder__max_categories',
 'columntransformer__cat__onehotencoder__min_frequency',
 'columntransformer__cat__onehotencoder__sparse_output',
 'randomforestclassifier__bootstrap',
 'randomforestclassifier__ccp_alpha',
 'randomforestclassifier__class_weight',
 'randomforestclassifier__criterion',
 'randomforestclassifier__max_depth',
 'randomforestclassifier__max_features',
 'randomforestclassifier__max_leaf_nodes',
 'randomforestclassifier__max_samples',
 'randomforestclassifier__min_impurity_decrease',
 'randomforestclassifier__min_samples_leaf',
 'randomforestclassifier__min_samples_split',
 'randomforestclassifier__min_weight_fraction_leaf',
 'randomforestclassifier__monotonic_cst',
 'randomforestclassifier__n_estimators',
 'randomforestclassifier__n_jobs',
 'randomforestclassifier__oob_score',
 'randomforestclassifier__random_state',
 'randomforestclassifier__verbose',
 'randomforestclassifier__warm_start']
```

Hyper Parameters

- Hyperparameters are parameters whose values control the learning process and determine the values of model parameters that a learning algorithm ends up learning. The prefix ‘hyper_’ suggests that they are ‘top-level’ parameters that control the learning process and the model parameters that result from it.

```python
param_grid = {
    'randomforestclassifier__class_weight': [None, 'balanced'],
    'randomforestclassifier__max_depth': [None, *range(3, 10, 2)],
    'randomforestclassifier__min_samples_leaf': list(range(3, 10, 2)),
    'randomforestclassifier__min_samples_split': list(range(5, 12, 2)),
    'randomforestclassifier__criterion': ['gini', 'entropy'],
    'randomforestclassifier__n_estimators': [10, 100, 500],
    'randomforestclassifier__max_features': [None, 'sqrt']
}
```

```python
model_pipeline_gridcv = GridSearchCV(model_pipeline, param_grid, n_jobs=-1)
model_pipeline_gridcv
```



Grid Search Cross Validation

- Exhaustive search over specified parameter values for an estimator.
- Helps us select the hyper parameters that gives the best model score

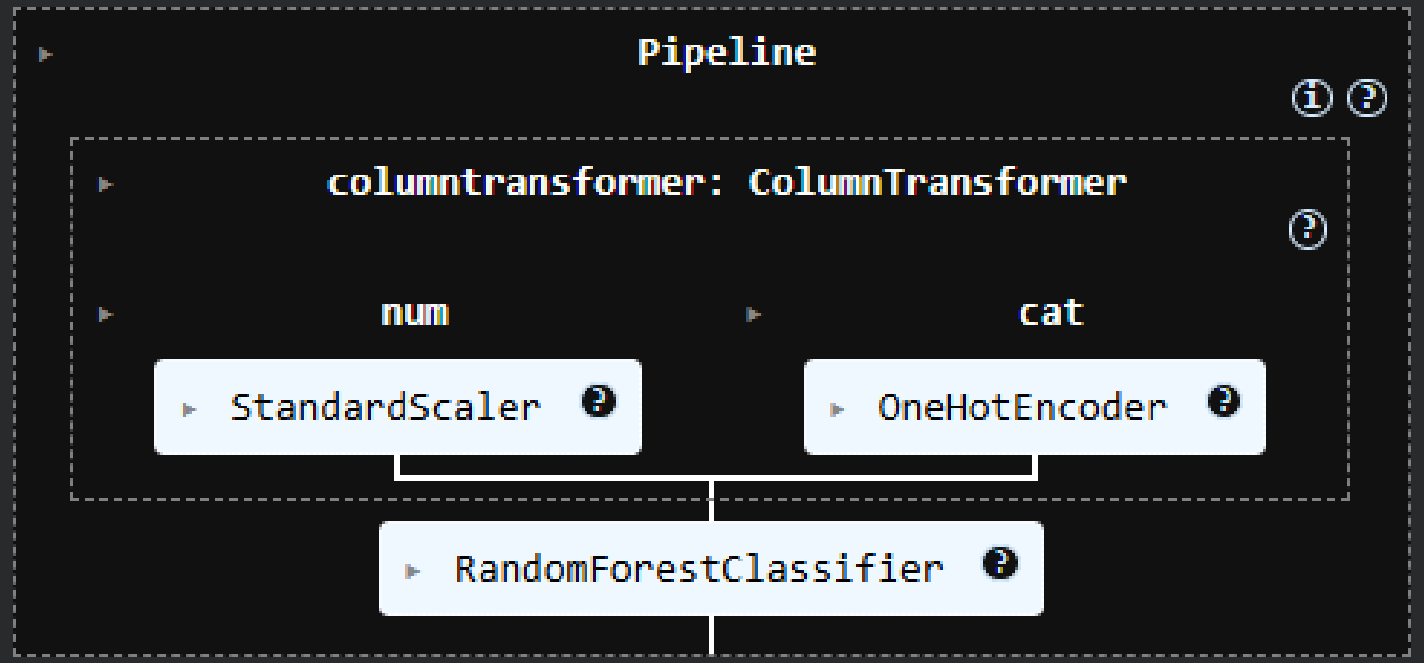
Best Parameters and Model Pipeline

- Use ‘best_params_’ parameter to see the best hyper parameters used
- Use ‘best_estimator_’ parameter to get the pipeline instance that uses the best hyper parameters

```
1 model_pipeline_gridcv.best_params_
```

```
{'randomforestclassifier__class_weight': 'balanced',
 'randomforestclassifier__criterion': 'entropy',
 'randomforestclassifier__max_depth': 9,
 'randomforestclassifier__max_features': 'sqrt',
 'randomforestclassifier__min_samples_leaf': 3,
 'randomforestclassifier__min_samples_split': 7,
 'randomforestclassifier__n_estimators': 100}
```

```
1 best_model_pipeline = model_pipeline_gridcv.best_estimator_
2 best_model_pipeline
```



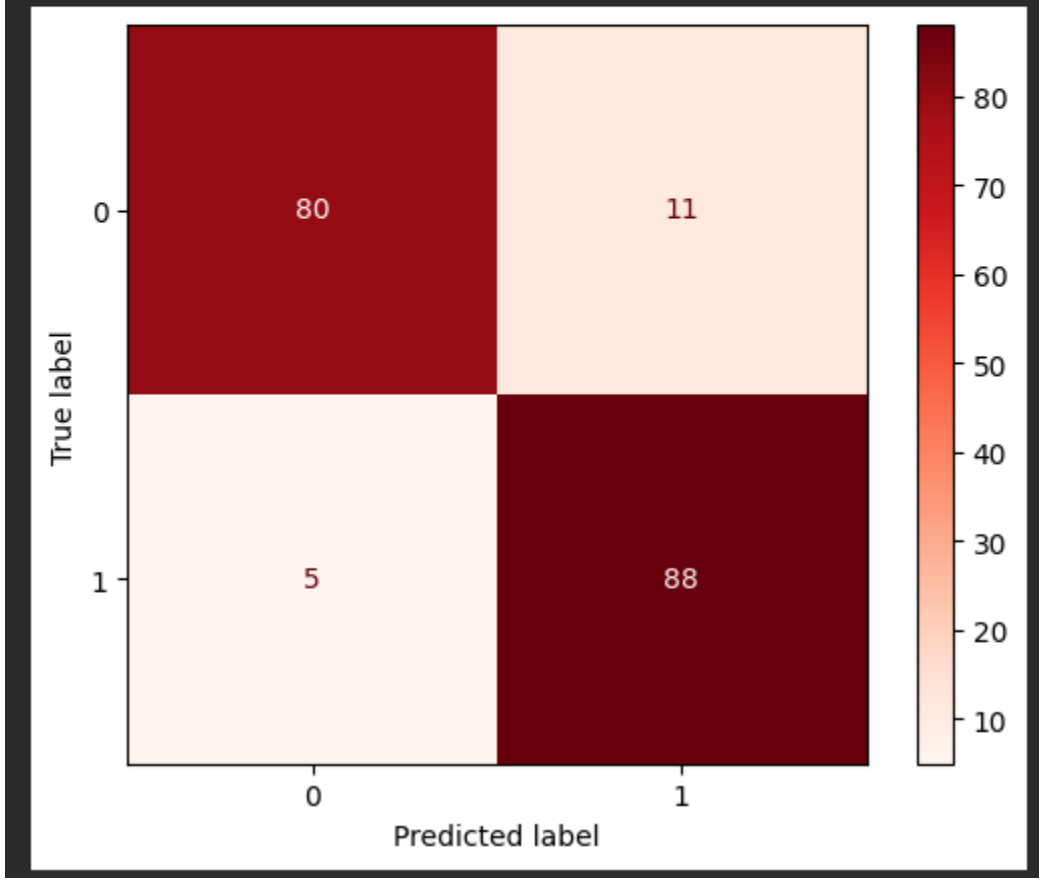
Model Evaluation

- Accuracy: How accurate our model prediction is
- Balanced Accuracy: The balanced accuracy in binary and multiclass classification problems to deal with imbalanced datasets. It is defined as the average of recall obtained on each class.
- Precision: The precision is intuitively the ability of the classifier not to label as positive a sample that is negative.
- Recall: The recall is intuitively the ability of the classifier to find all the positive samples.
- F1-Score: The F1 score can be interpreted as a harmonic mean of the precision and recall, where an F1 score reaches its best value at 1 and worst score at 0. The relative contribution of precision and recall to the F1 score are equal.
- Support: Number of occurrences in our testing data

```
Accuracy:           0.913
Balanced Accuracy:  0.913
              precision    recall  f1-score   support

           0       0.94      0.88      0.91        91
           1       0.89      0.95      0.92        93

    accuracy                           0.91       184
   macro avg       0.92      0.91      0.91       184
weighted avg       0.91      0.91      0.91       184
```



```python
new_test_x = pd.DataFrame({
    'Age': [19, 25, 33, 39, 45],
    'Sex': ['M', 'M', 'M', 'M', 'M'],
    'ChestPainType': ['ASY', 'ASY', 'ASY', 'ASY', 'ASY'],
    'RestingBP': [130, 130, 130, 130, 130],
    'Cholesterol': [200, 200, 200, 200, 200],
    'FastingBS': ['0', '0', '0', '0', '0'],
    'RestingECG': ['Normal', 'Normal', 'Normal', 'Normal', 'Normal'],
    'MaxHR': [130, 130, 130, 130, 130],
    'ExerciseAngina': ['N', 'N', 'N', 'N', 'N'],
    'Oldpeak': [1.0, 1.0, 1.0, 1.0, 1.0],
    'ST_Slope': ['Down', 'Flat', 'Up', 'Down', 'Up']
})
new_test_x
```

Show hidden output

```python
new_predictions = best_model_pipeline.predict(new_test_x)
new_predictions
```

```
array([0, 1, 0, 0, 0])
```

Model Usage

PROVIDE NEW INPUTS AND MAKE PREDICTIONS

Sources

- Dataset: https://www.kaggle.com/datasets/tan5577/heart-failure-dataset/data
- References:
  - Pipeline: https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html
  - Standard Scaler: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html, https://wycho.tistory.com/103, https://www.geeksforgeeks.org/machine-learning/standardscaler-minmaxscaler-and-robustscaler-techniques-ml/
  - One Hot Encoder: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html#sklearn.preprocessing.OneHotEncoder, https://www.geeksforgeeks.org/machine-learning/ml-one-hot-encoding/
  - Random Forest Classifier: https://medium.com/data-science/random-forest-explained-a-visual-guide-with-code-examples-9f736a6e1b3c, https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html#sklearn.ensemble.RandomForestClassifier
  - Hyperparameters: https://towardsdatascience.com/parameters-and-hyperparameters-aa609601a9ac/
  - Grid Search CV: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

THANK YOU